

CAPSTONE PROJECT REPORT

Web Application Penetration Testing using DVWA

1. Abstract

This project focuses on identifying and exploiting vulnerabilities in a deliberately vulnerable web application, DVWA (Damn Vulnerable Web Application). The goal of this project is to understand common web application security flaws and demonstrate how attackers can exploit them in real-world scenarios. Various attacks such as SQL Injection, Cross-Site Scripting (XSS), Command Injection, File Upload vulnerabilities, Brute Force attacks, and Cryptographic weaknesses were performed. The project highlights both offensive techniques and the importance of secure coding practices.

2. Introduction

Web applications are widely used in modern systems, making them a primary target for cyber attacks. Many applications suffer from vulnerabilities due to improper input validation, weak authentication mechanisms, and insecure configurations.

DVWA is a training platform designed to simulate real-world vulnerabilities. This project uses DVWA to perform penetration testing and analyze how attackers exploit weaknesses in web applications. It also helps in understanding defensive strategies to prevent such attacks.

3. Objective

- To understand common web vulnerabilities
 - To perform penetration testing in a controlled environment
 - To analyze security weaknesses in web applications
 - To gain hands-on experience with real attack techniques
 - To suggest mitigation strategies for each vulnerability
-

4. Tools Used

- Kali Linux
 - DVWA (Damn Vulnerable Web Application)
 - Apache Server
 - MySQL Database
 - Burp Suite (for request analysis)
 - John the Ripper (for password cracking)
-

5. Methodology

1. Set up DVWA in Kali Linux environment
2. Configure database and set security level to LOW
3. Identify vulnerabilities from DVWA modules
4. Perform attacks using manual and tool-based techniques
5. Capture screenshots of each successful attack
6. Analyze impact and provide mitigation techniques

6. Detailed Findings

6.1 SQL Injection

Description:

SQL Injection is a vulnerability where attackers manipulate database queries to gain unauthorized access to data.

Payload Used:

1' OR '1'='1

Advanced Payload:

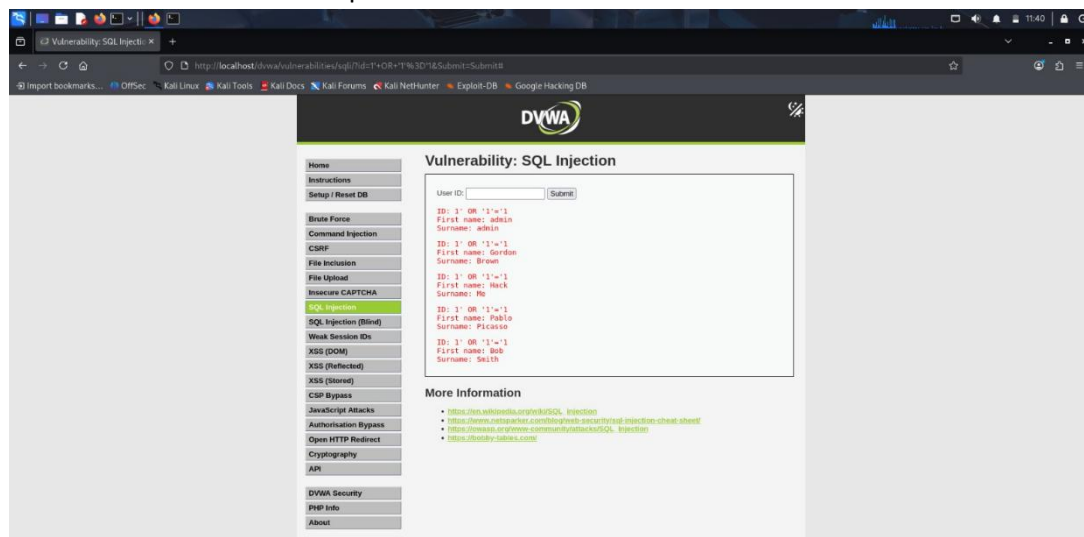
1' UNION SELECT user,password FROM users #

Observation:

The application returned multiple user records including usernames and hashed passwords.

Impact:

- Unauthorized access to database
- Data leakage
- Possible full database compromise



Mitigation:

- Use prepared statements
- Input validation
- Parameterized queries

6.2 Cross-Site Scripting (XSS)

Stored XSS

Payload:

```
<script>alert('Hacked')</script>
```

Observation:

The script executed and showed a popup alert.

Cookie Stealing XSS

Payload:

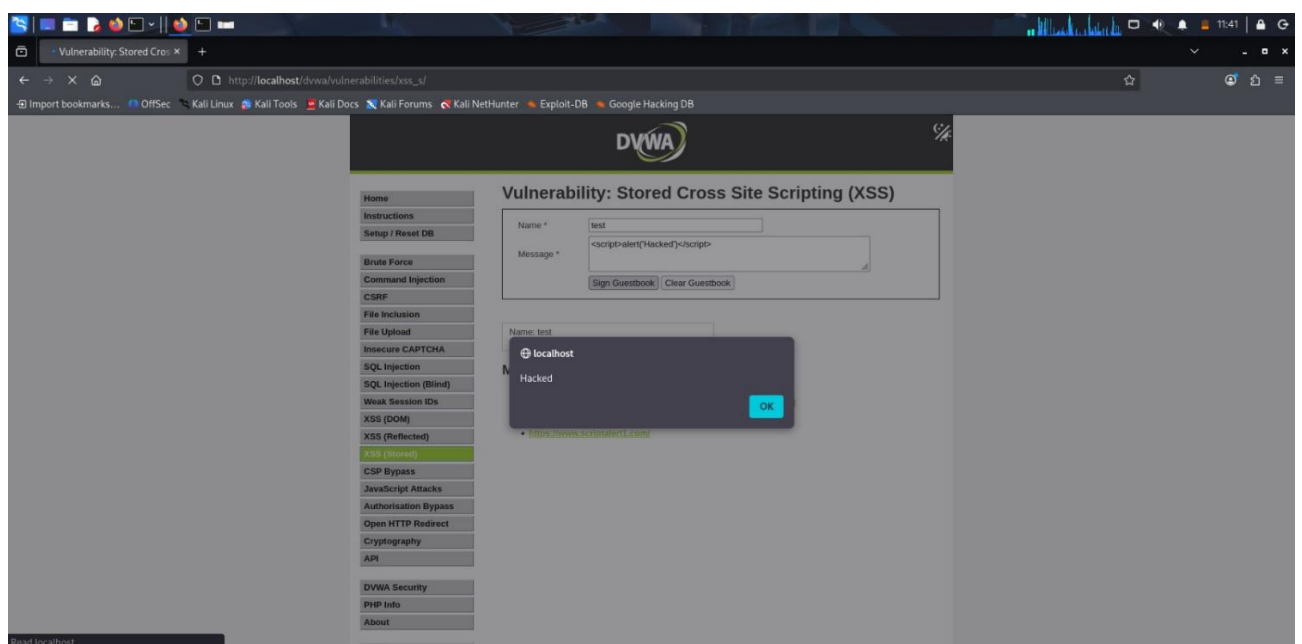
```
<script>document.write(document.cookie)</script>
```

Observation:

Session cookies were displayed on the page.

Impact:

- Session hijacking
- Unauthorized access



Mitigation:

- Input sanitization
 - Output encoding
 - Use Content Security Policy (CSP)
-

6.3 Command Injection

Payload:

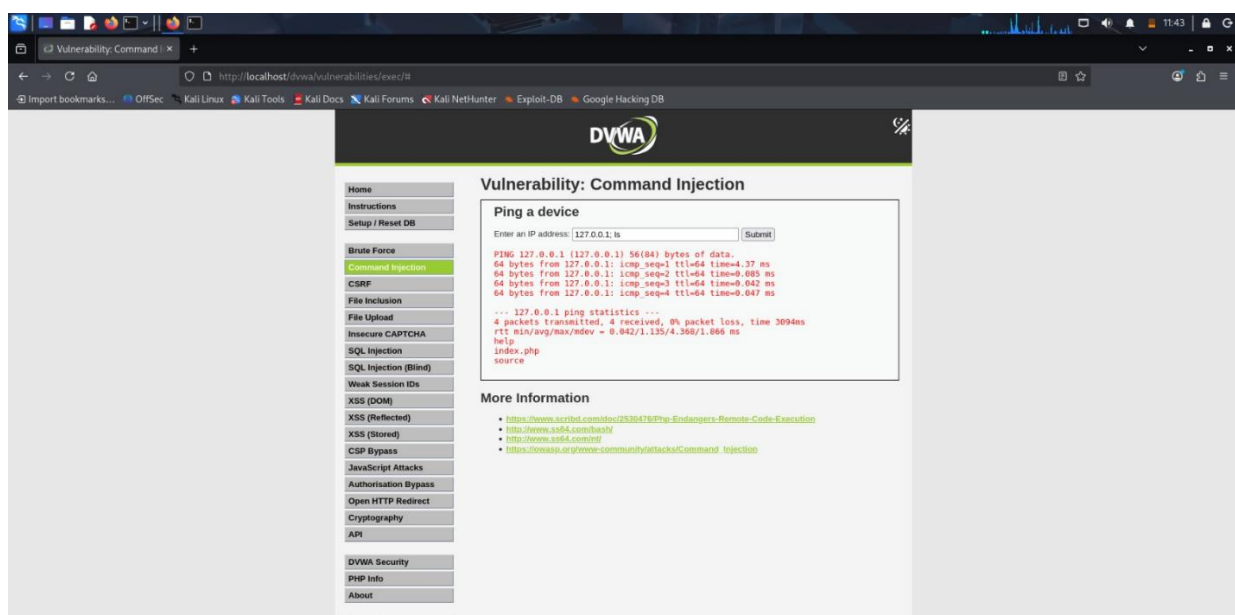
127.0.0.1; ls

Observation:

System commands were executed and file list was displayed.

Impact:

- Remote command execution
- Server compromise



Mitigation:

- Validate user input
- Avoid system command execution
- Use secure APIs

6.4 File Upload Vulnerability

Method:

Uploaded a PHP shell file.

Payload:

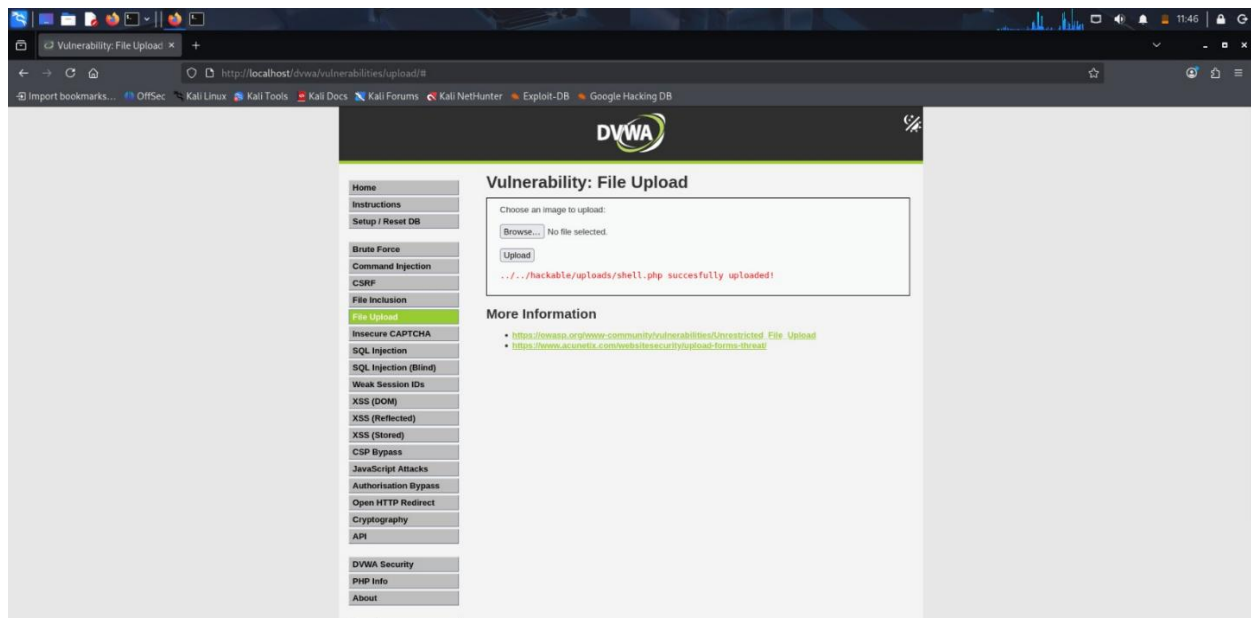
```
<?php system($_GET['cmd']); ?>
```

Observation:

Commands like ls and whoami were executed through browser.

Impact:

- Full server control
- Remote access



Mitigation:

- Restrict file types
- Disable execution permissions
- Validate uploads

6.5 Brute Force Attack

Method:

Tried multiple passwords manually.

Successful Credentials:

admin / password

Observation:

No account lock or CAPTCHA present.

Impact:

- Unauthorized login
- Weak authentication system

Mitigation:

- Account logout
 - CAPTCHA
 - Strong password policy
-

6.6 Cryptographic Weakness (MD5 Hash Cracking)

Hash Extracted:

5f4dcc3b5aa765d61d8327deb882cf99

Cracked Password:

password

Tool Used:

John the Ripper / Online tools

Observation:

Passwords stored using weak MD5 hashing.

Impact:

- Easy password recovery
- Data compromise

Mitigation:

- Use bcrypt or SHA-256 with salt
 - Avoid MD5
-

6.7 CSRF (Cross-Site Request Forgery)

Observation:

Password was changed using a crafted URL without user consent.

Impact:

- Unauthorized actions
- Account takeover

Mitigation:

- Use CSRF tokens
 - Validate requests
-

6.8 Directory Listing (Misconfiguration)

URL Accessed:

<http://localhost/dvwa/hackable/uploads/>

Observation:

All uploaded files were visible.

Impact:

- Exposure of sensitive files

Mitigation:

- Disable directory browsing
 - Proper server configuration
-

6.9 Clickjacking

Observation:

Application can be loaded inside an iframe.

Impact:

- User interaction hijacking

Mitigation:

- Use X-Frame-Options
 - Use CSP headers
-

7. Results

The DVWA application was found vulnerable to multiple critical attacks. All tested modules successfully demonstrated real-world vulnerabilities. The results clearly indicate that insecure coding practices can lead to serious security risks.

8. Conclusion

This project provided hands-on experience in identifying and exploiting web application vulnerabilities. It highlights the importance of secure development practices and proper security mechanisms. Understanding these vulnerabilities helps in building secure systems and preventing cyber attacks.

9. Project Scope

- Can be extended to real-world applications
 - Can integrate automated tools like Burp Suite Intruder
 - Can be used for SOC and defensive analysis
 - Can be enhanced with SIEM integration
-

10. References

- DVWA Official Documentation
- OWASP Top 10
- Kali Linux Documentation